CS2013 - Programación III
Fundamentos de Programación
Gestor de Drones de Reparto Urbano

Rubén Rivas Medina

Marzo 2026

Contexto

Una empresa de logística urbana está desarrollando un sistema para gestionar drones de reparto. Existen distintos tipos de drones, cada uno con capacidades específicas, y múltiples estrategias de navegación dependiendo del entorno (tráfico aéreo, zonas restringidas, clima, etc.).

El sistema debe permitir:

- gestionar drones heterogéneos
- asignar estrategias dinámicamente
- clonar configuraciones base
- optimizar el manejo de recursos mediante semántica de copia y movimiento

Objetivo

Implementar un sistema en C++ que haga uso de:

- Polimorfismo (herencia + virtual)
- Patrón Strategy
- Patrón Prototype (clone)
- Sobrecarga de operadores
- Constructores copy y move
- Administración de colecciones polimórficas

Requerimientos funcionales

1. Jerarquía base

Definir una clase abstracta:

- **`Drone`**
  - **`std::unique_ptr<Drone> clone() const`**
  - **`double calculate_range() const`**
  - **`set_strategy(std::shared_ptr<NavigationStrategy> strategy)`**
  - **`Route plan_route(const DeliveryRequest& request) const`**
  - **`bool can_execute_delivery(const DeliveryRequest& request) const`**
  - **`void execute_delivery(const DeliveryRequest& request)`**

Consideraciones:

- **`calculate_range()`** debe retornar un valor numérico no negativo que represente la autonomía o alcance operativo del drone.
- Para evitar ambigüedad, asumir que cada drone almacena internamente una autonomía base expresada en metros.

- Esa autonomía base debe definirse en el constructor de cada clase concreta o derivarse de atributos internos claramente modelados por el estudiante.
- Si el constructor recibe un valor como **`10.0`** o **`25.0`**, ese valor debe interpretarse como la autonomía base inicial del drone.
- **`calculate_range()`** debe obtener el alcance a partir de ese estado interno y retornar el valor final de autonomía disponible.
- Una implementación mínima y totalmente válida para este enunciado es: **`range = max(0.0, autonomia_base)`**.
- Si el estudiante decide usar una fórmula más elaborada, esta debe estar claramente documentada y seguir siendo determinista, no negativa y consistente entre llamadas mientras el estado del objeto no cambie.
- Ejemplo: si **`LightDrone d1(10.0, strategy1);`**, entonces una implementación válida es que **`d1.calculate_range()`** retorne **`10.0`**.
- **`set_strategy(...)`** debe permitir reemplazar la estrategia de navegación en tiempo de ejecución.
- **`plan_route(...)`** debe delegar el cálculo de la ruta a la estrategia actual del drone para una solicitud de entrega concreta.
- **`can_execute_delivery(...)`** debe indicar si el drone puede realizar la entrega solicitada considerando al menos la distancia estimada y el peso de la carga.
- La validación mínima de **`can_execute_delivery(...)`** debe considerar simultáneamente:
- **`request.payload_weight > 0.0`**
- **`request.payload_weight <= capacidad_maxima_del_tipo_concreto`**
- **`plan_route(request).estimated_distance <= calculate_range()`**
- **`execute_delivery(...)`** debe ejecutar una entrega concreta y debe depender del tipo de drone, de la ruta calculada y de la compatibilidad de la solicitud con sus capacidades.
- **`execute_delivery(...)`** no debe ejecutar la entrega si **`can_execute_delivery(...)`** retorna **`false`**.
- Una entrega solo debe considerarse viable si la **`Route`** calculada es compatible con la autonomía retornada por **`calculate_range()`**.
- La clase base debe permitir trabajar polimórficamente con cualquier tipo de drone sin conocer su implementación concreta.

Implementar al menos:

- **`LightDrone`**
- **`HeavyDrone`**

Se espera que ambos tipos difieran en al menos uno de estos aspectos:

- autonomía
- capacidad de carga
- comportamiento de entrega
- criterios internos de operatividad
- Para esta versión del enunciado, fijar explícitamente la especialización por capacidad de carga.
- **`LightDrone`** debe admitir como máximo **`2.0`** kg de carga útil.
- **`HeavyDrone`** debe admitir como máximo **`8.0`** kg de carga útil.
- En consecuencia, **`can_execute_delivery(...)`** debe rechazar cualquier solicitud cuyo **`payload_weight`** exceda el límite del tipo concreto.
- Esta diferencia debe reflejarse en el comportamiento observable de los objetos y justifica la existencia de dos clases concretas distintas.

2. Strategy: Navegación

Definir:

- **`Location`**
  - **`double x`**
  - **`double y`**
  - **`double z`**
- **`Route`**
  - **`std::string description`**
  - **`double estimated_distance`**
- **`DeliveryRequest`**
  - **`std::string id`**
  - **`Location origin`**
  - **`Location destination`**
  - **`double payload_weight`**
- **`NavigationStrategy`**
  - **`Route calculate_route(const DeliveryRequest& request) const`**

Consideraciones:

- **`Location`** representa una posición espacial del drone usada como origen o destino.
- **`x`**, **`y`** y **`z`** representan coordenadas espaciales, donde **`z`** corresponde a la altitud.
- **`Route`** representa el resultado del cálculo de navegación e incluye una descripción legible y una distancia estimada.
- **`DeliveryRequest`** representa una solicitud de entrega y agrupa el origen, el destino y el peso de la carga a transportar.
- **`payload_weight`** representa la carga útil solicitada y debe interpretarse en kilogramos.
- Para este enunciado, una solicitud válida debe cumplir **`payload_weight > 0.0`**.
- **`calculate_route(...)`** debe retornar un objeto **`Route`** completamente válido para la solicitud recibida.
- Asumir que las coordenadas de **`Location`** están expresadas en metros.
- **`estimated_distance`** debe expresarse en metros.
- **`description`** no debe ser vacía y **`estimated_distance`** debe ser un valor positivo.
- Para este enunciado, tanto **`FastRoute`** como **`SafeRoute`** deben partir de la distancia euclidiana 3D entre **`origin`** y **`destination`**.
- La distancia base debe calcularse como **`sqrt(dx * dx + dy * dy + dz * dz)`**, donde **`dx = destination.x - origin.x`**, **`dy = destination.y - origin.y`** y **`dz = destination.z - origin.z`**.
- **`FastRoute`** debe retornar esa distancia euclidiana directa como **`estimated_distance`**.
- **`SafeRoute`** debe retornar una ruta basada en la misma distancia euclidiana, pero multiplicada por un factor de seguridad **`1.2`**.
- En consecuencia, una fórmula válida para **`SafeRoute`** es **`estimated_distance = distancia_euclidiana * 1.2`**.
- La **`description`** debe permitir distinguir la estrategia usada; por ejemplo, una ruta rápida puede describirse como directa y una ruta segura como ajustada por factor de seguridad.
- Si se desea estimar tiempo de vuelo, ese cálculo debe depender además de características del drone, como su velocidad de crucero, por lo que no forma parte obligatoria de **`NavigationStrategy`** en este enunciado.

- Estrategias distintas deben poder producir rutas distintas para una misma **`DeliveryRequest`**.
- El objetivo del patrón es que el comportamiento de navegación pueda cambiar sin reconstruir el drone.

Implementaciones:

- **`FastRoute`**
- **`SafeRoute`**

Cada drone debe agregar una estrategia y permitir cambiarla dinámicamente.

Se espera que estrategias distintas produzcan resultados observables distintos para un mismo escenario de navegación.

3. Prototype
Cada drone debe implementar:

- **`std::unique_ptr<Drone> clone() const`**

Debe garantizar:

- preservación del tipo dinámico
- copia correcta del estado
- independencia entre original y clon frente a cambios posteriores

4. Sobrecarga de operadores
Implementar:

- **`operator<`** para comparar drones por autonomía
- **`operator<<`** para imprimir el estado de un drone en **`std::ostream`**
- **`operator bool`** para indicar si un drone está operativo

Consideraciones:

- **`operator<`** debe definir un criterio consistente y estable, apto para ordenamiento.
- **`operator<<`** debe exponer información suficiente para inspeccionar el estado del objeto, como tipo, autonomía y estrategia actual.
- **`operator bool`** debe tener una regla explícita de dominio; por ejemplo, considerar operativo a un drone con autonomía positiva y configuración válida.

5. Copy / Move semantics
Cada clase concreta debe implementar:

- constructor de copia
- constructor de movimiento
- operador de asignación (copy/move)

Consideraciones:

- Las operaciones de copia deben preservar el estado observable sin compartir recursos mutables de forma accidental.
- Las operaciones de movimiento deben transferir recursos de forma segura y dejar el objeto origen en un estado válido según la implementación.

6. Centro de operaciones
Definir una clase **`OperationsCenter`** que actúe como administrador de una colección polimórfica de drones.

Debe permitir como mínimo:

- almacenar múltiples **`std::unique_ptr<Drone>`**
- agregar drones al centro mediante **`std::move`**
- despachar entregas polimórficas sobre todos los drones
- mostrar el estado de los drones almacenados
- consultar cuántos drones hay registrados
- manejar correctamente el caso de centro vacío

Operaciones mínimas sugeridas:

- **`void add_drone(std::unique_ptr<Drone> drone)`**
- **`void dispatch_delivery(const DeliveryRequest& request)`**
- **`void dispatch_deliveries(const std::vector<DeliveryRequest>& requests)`**
- **`void show_drones() const`**
- **`size_t drone_count() const`**

Consideraciones:

- **`add_drone(...)`** debe recibir ownership exclusivo mediante **`std::move`**.
- **`dispatch_delivery(...)`** debe intentar asignar una solicitud de entrega a un drone compatible.
- La compatibilidad debe evaluarse usando **`can_execute_delivery(...)`**, por lo que la carga útil participa directamente en la decisión de despacho.
- **`dispatch_deliveries(...)`** debe procesar múltiples solicitudes de entrega y permitir que distintas entregas tengan distintos orígenes y destinos.
- El criterio de asignación puede ser simple, siempre que esté claramente definido; por ejemplo, el primer drone compatible o el de mayor autonomía disponible.
- **`show_drones()`** debe facilitar la inspección del estado actual de los drones almacenados.
- El centro debe comportarse correctamente incluso cuando la colección esté vacía.

Restricciones

- Usar **`std::unique_ptr`** para drones
- Usar **`std::shared_ptr`** (u otra decisión explícita) para estrategias
- No usar librerías externas
- Separar **`.h`** y **`.cpp`**
- Evitar memory leaks

Casos de prueba

Los ejemplos de esta sección pueden asumir, salvo indicación contraria, el siguiente setup común:

```
auto strategy1 = std::make_shared<FastRoute>();
auto strategy2 = std::make_shared<SafeRoute>();
DeliveryRequest request1{
    "REQ-001",
    {0.0, 0.0, 10.0},
    {100.0, 50.0, 20.0},
    2.0
};
DeliveryRequest request2{
    "REQ-002",
    {10.0, 5.0, 12.0},
    {80.0, 20.0, 18.0},
    5.0
```

```
};
LightDrone d1(10.0, strategy1);
HeavyDrone d2(25.0, strategy2);
```

Interpretación explícita del setup:

- **`request1`** tiene **`payload_weight = 2.0`**, por lo que puede ser compatible con **`LightDrone`** o **`HeavyDrone`** si además la ruta está dentro de la autonomía.
- **`request2`** tiene **`payload_weight = 5.0`**, por lo que debe ser rechazado por **`LightDrone`** y solo puede ser compatible con **`HeavyDrone`** si además la ruta está dentro de la autonomía.

Sección A: Pruebas obligatorias

CP1: Creación de drones

- Crear **`LightDrone`** y **`HeavyDrone`**
- Validar construcción correcta y autonomía válida

Ejemplo:

```
auto strategy1 = std::make_shared<FastRoute>();
auto strategy2 = std::make_shared<SafeRoute>();
LightDrone d1(10.0, strategy1);
HeavyDrone d2(25.0, strategy2);

std::cout << d1.calculate_range() << std::endl;
std::cout << d2.calculate_range() << std::endl;

assert(d1.calculate_range() > 0.0);
assert(d2.calculate_range() > 0.0);
```

Resultado esperado:

```
Ambos objetos se construyen sin errores.
Se imprimen dos valores numéricos positivos de autonomía.
Las aserciones se cumplen.
```

CP1B: Compatibilidad por carga útil

- Validar el uso de **`payload_weight`** en **`can_execute_delivery(...)`**
- Verificar que **`LightDrone`** y **`HeavyDrone`** respondan distinto ante cargas distintas

Ejemplo:

```
LightDrone light(10.0, strategy1);
HeavyDrone heavy(25.0, strategy2);

assert(light.can_execute_delivery(request1));
assert(!light.can_execute_delivery(request2));
assert(heavy.can_execute_delivery(request1));
assert(heavy.can_execute_delivery(request2));
```

Resultado esperado:

```
`request1` es compatible con ambos tipos si la ruta está dentro de la autonomía.
`request2` no es compatible con `LightDrone` porque excede su carga útil máxima.
`request2` puede ser compatible con `HeavyDrone` si la ruta está dentro de la
autonomía.
La carga útil queda validada y usada explícitamente en
`can_execute_delivery(...)`.
```

CP2: Polimorfismo

- Usar **`std::unique_ptr<Drone>`**
- Validar despacho dinámico correcto

Ejemplo:

```
std::vector<std::unique_ptr<Drone>> drones;
drones.push_back(std::make_unique<LightDrone>(10.0, strategy1));
drones.push_back(std::make_unique<HeavyDrone>(25.0, strategy2));

assert(typeid(*drones[0]) == typeid(LightDrone));
assert(typeid(*drones[1]) == typeid(HeavyDrone));

for (const auto& drone : drones) {
    drone->execute_delivery(request1);
    std::cout << *drone << std::endl;
}
```

Resultado esperado:

```
Se invoca la implementación correspondiente a cada tipo concreto.
Se imprime el estado de ambos drones.
Se verifica que la colección preserve el tipo dinámico de cada objeto.
No se requieren casts ni condicionales por tipo.
```

CP3: Estrategia inicial

- Asignar **`FastRoute`** y **`SafeRoute`**
- Validar diferencias en comportamiento

Ejemplo:

```
FastRoute fast;
SafeRoute safe;

Route fast_route = fast.calculate_route(request1);
Route safe_route = safe.calculate_route(request1);

std::cout << fast_route.description << std::endl;
std::cout << safe_route.description << std::endl;

assert(!fast_route.description.empty());
assert(!safe_route.description.empty());
assert(fast_route.estimated_distance > 0.0);
```

```
assert(safe_route.estimated_distance > 0.0);
assert(safe_route.estimated_distance > fast_route.estimated_distance);
```

Resultado esperado:

```
Ambas estrategias generan un objeto `Route` válido.
Las rutas tienen descripción no vacía y distancia positiva.
`FastRoute` usa la distancia euclidiana directa.
`SafeRoute` usa la misma base, pero con el factor `1.2`, por lo que su distancia
es mayor.
```

CP4: Cambio dinámico de estrategia

- Cambiar estrategia en runtime
- Validar cambio de comportamiento

Ejemplo:

```
LightDrone drone(10.0, std::make_shared<FastRoute>());
Route before = drone.plan_route(request1);

drone.set_strategy(std::make_shared<SafeRoute>());
Route after = drone.plan_route(request1);

std::cout << before.description << std::endl;
std::cout << after.description << std::endl;
assert(before.description != after.description
    || before.estimated_distance != after.estimated_distance);

drone.execute_delivery(request1);
```

Resultado esperado:

```
La ruta calculada antes y después del cambio no es idéntica.
La nueva ejecución usa la estrategia recién asignada.
El cambio ocurre sin reconstruir el drone.
```

CP5: Clonación

- Clonar un drone
- Validar igualdad inicial e independencia

Ejemplo:

```
auto clone = d1.clone();
std::ostringstream original_before;
std::ostringstream clone_before;
original_before << d1;
clone_before << *clone;

assert(typeid(*clone) == typeid(d1));
assert(original_before.str() == clone_before.str());
```

```
clone->set_strategy(std::make_shared<SafeRoute>());

std::ostringstream original_after;
std::ostringstream clone_after;
original_after << d1;
clone_after << *clone;

std::cout << original_after.str() << std::endl;
std::cout << clone_after.str() << std::endl;
assert(original_after.str() != clone_after.str());
```

Resultado esperado:

```
El clon conserva el tipo dinámico y el estado inicial.
La comprobación de tipo dinámico se cumple.
La representación inicial del clon coincide con la del original.
Tras cambiar el clon, el estado observable del original permanece intacto.
```

CP6: Constructor de copia

- Copiar un drone
- Validar independencia del original

Ejemplo:

```
LightDrone original(10.0, strategy1);
LightDrone copy = original;

std::ostringstream original_before;
std::ostringstream copy_before;
original_before << original;
copy_before << copy;

assert(copy.calculate_range() == original.calculate_range());
assert(original_before.str() == copy_before.str());

copy.set_strategy(std::make_shared<SafeRoute>());

std::ostringstream original_after;
std::ostringstream copy_after;
original_after << original;
copy_after << copy;

std::cout << original_after.str() << std::endl;
std::cout << copy_after.str() << std::endl;
assert(original_after.str() != copy_after.str());
```

Resultado esperado:

```
copy inicia con el mismo estado observable que original.
La comparación inicial de autonomía coincide.
Tras modificar copy, el estado observable de original no cambia.
```

CP7: Constructor de movimiento

- Mover un drone
- Validar transferencia de recursos

Ejemplo:

```
LightDrone original(10.0, strategy1);
double previous_range = original.calculate_range();
LightDrone moved = std::move(original);

std::cout << moved << std::endl;
assert(moved.calculate_range() == previous_range);
assert(moved.calculate_range() > 0.0);
```

Resultado esperado:

```
moved conserva los recursos válidos.
La autonomía transferida a `moved` coincide con la del objeto antes del move.
El objeto movido queda en un estado seguro según la implementación.
```

CP8: Operador <

- Comparar drones
- Validar orden correcto

Ejemplo:

```
LightDrone d1(10.0, strategy1);
HeavyDrone d2(25.0, strategy2);
bool result = d1 < d2;

std::cout << std::boolalpha << result << std::endl;
assert(result);
```

Resultado esperado:

```
Se imprime `true`.
La aserción se cumple.
El criterio de comparación es consistente en evaluaciones repetidas.
```

CP9: Operador <<

- Imprimir un drone en consola
- Validar formato de salida y consistencia del estado

Ejemplo:

```
std::cout << d1 << std::endl;
std::ostringstream out;
out << d1;
assert(!out.str().empty());
```

Resultado esperado:

```
La salida incluye información relevante del drone, como tipo,
autonomía y estrategia actual.
La impresión ocurre sin errores.
La representación generada no es vacía.
```

CP10: Operador bool

- Evaluar drones en contexto booleano
- Validar comportamiento para drones operativos y no operativos

Ejemplo:

```
LightDrone active(10.0, strategy1);
LightDrone inactive(0.0, strategy1);

if (active) {
    std::cout << "Drone operativo\n";
}

assert(static_cast<bool>(active) == true);
assert(static_cast<bool>(inactive) == false);
```

Resultado esperado:

```
Se imprime `Drone operativo`.
La conversión booleana retorna true para un drone operativo.
La conversión booleana retorna false para un drone sin autonomía.
```

CP11: Registro en centro

- Insertar drones con **`std::move`**
- Validar ownership correcto

Ejemplo:

```
OperationsCenter center;
auto d1 = std::make_unique<LightDrone>(10.0, strategy1);
center.add_drone(std::move(d1));

assert(d1 == nullptr);
assert(center.drone_count() == 1);
```

Resultado esperado:

```
El centro almacena el drone.
El unique_ptr original queda vacío y no hay copias accidentales.
La cantidad de drones registrada es 1.
```

CP12: Despacho en centro

- Despachar entregas desde un origen hacia un destino
- Validar comportamiento polimórfico

Ejemplo:

```
OperationsCenter center;

center.add_drone(std::make_unique<LightDrone>(10.0, strategy1));
center.add_drone(std::make_unique<HeavyDrone>(25.0, strategy2));
std::cout << center.drone_count() << std::endl;

std::vector<DeliveryRequest> requests{request1, request2};
center.dispatch_deliveries(requests);
```

Resultado esperado:

```
Se imprime `2`.
Se recorren las solicitudes entregadas al centro para despacho.
Cada entrega se intenta asignar a un drone compatible usando el criterio
definido.
```

CP13: Centro vacío

- Despachar sin drones
- Validar que no falla

Ejemplo:

```
OperationsCenter center;
std::cout << center.drone_count() << std::endl;

std::vector<DeliveryRequest> requests{request1, request2};
center.dispatch_deliveries(requests);
```

Resultado esperado:

```
Se imprime `0`.
No ocurre crash, excepción inesperada ni acceso inválido.
Opcionalmente puede mostrarse un mensaje informativo.
```

Sección B: Pruebas recomendadas

CP14: Clone + Strategy

- Clonar y cambiar estrategia en el clon
- Validar independencia

Ejemplo:

```
LightDrone original(10.0, std::make_shared<FastRoute>());
auto clone = original.clone();

std::ostringstream original_before;
std::ostringstream clone_before;
original_before << original;
clone_before << *clone;
```

```
assert(original_before.str() == clone_before.str());

clone->set_strategy(std::make_shared<SafeRoute>());

std::ostringstream original_after;
std::ostringstream clone_after;
original_after << original;
clone_after << *clone;

std::cout << original_after.str() << std::endl;
std::cout << clone_after.str() << std::endl;
assert(original_after.str() != clone_after.str());
```

Resultado esperado:

```
El clon inicia con el mismo estado observable que el original.
Después de cambiar la estrategia del clon, sus estados divergen.
El original conserva su estrategia inicial.
```

CP15: Assignment operators

- Probar asignación por copia y por movimiento
- Validar consistencia e integridad del estado

Ejemplo:

```
LightDrone source(10.0, std::make_shared<FastRoute>());
LightDrone copy_target(5.0, std::make_shared<SafeRoute>());
copy_target = source;

std::ostringstream source_state;
std::ostringstream copy_state;
source_state << source;
copy_state << copy_target;

assert(source_state.str() == copy_state.str());

LightDrone move_source(20.0, std::make_shared<SafeRoute>());
double move_range = move_source.calculate_range();
LightDrone move_target(1.0, std::make_shared<FastRoute>());
move_target = std::move(move_source);

assert(move_target.calculate_range() == move_range);
std::cout << copy_target << std::endl;
std::cout << move_target << std::endl;
```

Resultado esperado:

```
La asignación por copia deja ambos objetos con el mismo estado observable.
La asignación por movimiento transfiere correctamente el estado al destino.
Los objetos destino quedan válidos y consistentes tras ambas operaciones.
```

CP16: Comparación entre tipos

- Comparar drones de distinto tipo

Ejemplo:

```
LightDrone light(15.0, std::make_shared<FastRoute>());
HeavyDrone heavy(30.0, std::make_shared<SafeRoute>());

bool light_less = light < heavy;
bool heavy_less = heavy < light;

std::cout << std::boolalpha << light_less << std::endl;
std::cout << std::boolalpha << heavy_less << std::endl;
assert(light_less != heavy_less);
```

Resultado esperado:

```
La comparación entre tipos distintos sigue el mismo criterio de autonomía.
Solo una de las comparaciones opuestas resulta verdadera.
El operador mantiene consistencia independientemente del tipo concreto.
```

CP17: Colección mixta

- Insertar clones y originales
- Validar independencia

Ejemplo:

```
OperationsCenter center;
auto original = std::make_unique<LightDrone>(10.0,
std::make_shared<FastRoute>());
auto clone = original->clone();

clone->set_strategy(std::make_shared<SafeRoute>());

center.add_drone(std::move(original));
center.add_drone(std::move(clone));

assert(center.drone_count() == 2);
center.show_drones();
```

Resultado esperado:

```
La colección acepta originales y clones en la misma estructura polimórfica.
Ambos objetos permanecen independientes después de insertarlos.
El centro registra correctamente los 2 drones.
```

Sección C: Pruebas avanzadas

CP18: Ordenamiento

- Ordenar drones por autonomía

Ejemplo:

```
std::vector<std::unique_ptr<Drone>> drones;
drones.push_back(std::make_unique<HeavyDrone>(25.0, strategy2));
drones.push_back(std::make_unique<LightDrone>(10.0, strategy1));
drones.push_back(std::make_unique<HeavyDrone>(40.0, strategy2));

std::sort(drones.begin(), drones.end(),
    [](const std::unique_ptr<Drone>& a, const std::unique_ptr<Drone>& b) {
        return *a < *b;
    });

assert(drones[0]->calculate_range() <= drones[1]->calculate_range());
assert(drones[1]->calculate_range() <= drones[2]->calculate_range());

for (const auto& drone : drones) {
    std::cout << drone->calculate_range() << std::endl;
}
```

Resultado esperado:

```
La colección queda ordenada de menor a mayor autonomía.
Las aserciones de orden se cumplen.
Se imprimen las autonomías en secuencia no decreciente.
```

CP19: Flujo completo

- Crear drones
- Asignar estrategias
- Clonar
- Imprimir estado
- Evaluar operatividad
- Insertar en centro
- Despachar entregas

Ejemplo:

```
auto strategy1 = std::make_shared<FastRoute>();
auto strategy2 = std::make_shared<SafeRoute>();
DeliveryRequest request1{
    "REQ-001",
    {0.0, 0.0, 10.0},
    {100.0, 50.0, 20.0},
    2.0
};
DeliveryRequest request2{
    "REQ-002",
    {10.0, 5.0, 12.0},
    {80.0, 20.0, 18.0},
    5.0
};

auto d1 = std::make_unique<LightDrone>(10.0, strategy1);
auto d2 = std::make_unique<HeavyDrone>(25.0, strategy2);
auto d3 = d1->clone();
```

```
d3->set_strategy(std::make_shared<SafeRoute>());

std::ostringstream original_state;
std::ostringstream clone_state;
original_state << *d1;
clone_state << *d3;

assert(original_state.str() != clone_state.str());
assert(static_cast<bool>(*d1) == true);
assert(static_cast<bool>(*d2) == true);

OperationsCenter center;
center.add_drone(std::move(d1));
center.add_drone(std::move(d2));
center.add_drone(std::move(d3));

assert(center.drone_count() == 3);
center.show_drones();

std::vector<DeliveryRequest> requests{request1, request2};
center.dispatch_deliveries(requests);
```

Resultado esperado:

```
El clon queda con un estado distinto al del original tras cambiar su estrategia.
Los drones operativos evalúan a true.
El centro registra 3 drones.
Las solicitudes de entrega se procesan sin errores sobre la colección registrada.
```

CP20: Múltiples movimientos

- Encadenar **`std::move`**
- Validar estabilidad

Ejemplo:

```
auto source = std::make_unique<LightDrone>(10.0, strategy1);
std::unique_ptr<Drone> step1 = std::move(source);
std::unique_ptr<Drone> step2 = std::move(step1);

assert(source == nullptr);
assert(step1 == nullptr);
assert(step2 != nullptr);
assert(step2->calculate_range() > 0.0);

std::cout << *step2 << std::endl;
```

Resultado esperado:

```
Los punteros intermedios quedan nulos después de cada move.
El último puntero conserva un objeto válido.
La autonomía y el estado observable del objeto final siguen siendo consistentes.
```

Ejemplo de uso esperado

```
int main() {
    auto strategy1 = std::make_shared<FastRoute>();
    auto strategy2 = std::make_shared<SafeRoute>();
    DeliveryRequest request1{
        "REQ-001",
        {0.0, 0.0, 10.0},
        {100.0, 50.0, 20.0},
        2.0
    };
    DeliveryRequest request2{
        "REQ-002",
        {10.0, 5.0, 12.0},
        {80.0, 20.0, 18.0},
        5.0
    };

    auto d1 = std::make_unique<LightDrone>(10.0, strategy1);
    auto d2 = std::make_unique<HeavyDrone>(25.0, strategy2);

    auto d3 = d1->clone();
    d3->set_strategy(std::make_shared<SafeRoute>());

    if (*d1 < *d2) {
        std::cout << "d2 tiene mayor autonomia\n";
    }

    std::cout << *d1 << std::endl;

    if (*d2) {
        std::cout << "d2 operativo\n";
    }

    OperationsCenter center;
    center.add_drone(std::move(d1));
    center.add_drone(std::move(d2));
    center.add_drone(std::move(d3));

    center.dispatch_deliveries({request1, request2});

    return 0;
}
```